

A Holistic Source-To-Source Compilation Approach for Partitioning and Optimizing C/C++ Applications for CPU-FPGA Systems

Tiago Santos^{*,†,1,2}, João Bispo^{*,1},
João M. P. Cardoso^{*,1}

** Faculty of Engineering of the University of Porto, s/n, R. Dr. Roberto Frias,
4200-465 Porto Portugal*

† INESC-TEC, Rua Dr. Roberto Frias, 4200-465 Porto Portugal

ABSTRACT

A common approach to improve performance is accelerating critical code regions using FPGAs. This approach often involves two processes: hardware/software partitioning, which identifies code regions to offload to the FPGA; and optimizing those regions (e.g., through HLS directives). As both processes are separate and usually applied in sequence, the interplay between them is undeveloped: there is a holistic lack of understanding regarding how the decisions in one process can positively influence the decisions of the other. This paper presents our work-in-progress for an approach that combines partitioning and optimization into a single holistic process. First, from the input C/C++ application, our source-to-source compiler builds a task-based intermediate representation. Then, a greedy algorithm builds clusters of tasks and assigns each cluster to either hardware (FPGA) or software (CPU). The algorithm iteratively refines the clusters and respective offloading decisions by dealing with two complementary targets: a) it minimizes the communication costs between clusters by assigning tasks that work with shared data to the same cluster; b) it reduces the global execution time by applying code optimizations to the tasks in each cluster. We conclude by reflecting on some preliminary experimental results obtained using a motivating example, as well as on the next steps of this project.

KEYWORDS: Source-to-Source compiler, Hardware/Software Partitioning, High-level Synthesis, FPGA

1 Introduction

The demand for adaptive computing is high, from data centers and cloud infrastructures capable of processing high volumes of data, to small embedded devices performing computations on the edge. By using hardware accelerators specialized to run critical regions of an

¹E-mail: {tiagolascasas,jbispo,jmpc}@fe.up.pt

²Supported by PhD grant "2021.07324.BD", financed by Fundação para a Ciência e Tecnologia (FCT)

application, we may achieve high performance in terms of throughput and execution time, while also meeting strict thermal and power constraints.

One concrete example of an adaptive computing platform comes in the form of heterogeneous CPU and FPGA systems. These systems allow for the execution of a C/C++ application on a CPU, while offloading one or more of its regions to an FPGA accelerator. This approach is particularly enticing for the above scenarios, as modern FPGAs range from server-grade devices on PCIe cards, making them suitable for high throughput cloud infrastructures, to embedded components on SoCs, which makes them apt for deployment on the edge or any other context with significant size, thermal and power constraints.

However, making a traditional C/C++ software application suitable for partial hardware acceleration presents its own set of challenges. The first of these is known as the Hardware/Software Partitioning problem, and consists of deciding which code regions should be offloaded to the FPGA. This often involves modelling the application as a set of tasks, with each task associated with performance metrics and resource usages on both the software (CPU) and hardware (FPGA), as well as the communication costs between the tasks [HYH].

After we solve this problem and arrive at an adequate partitioning, we need to further optimize the tasks that are set to be offloaded: we can convert C/C++ code into a hardware design by using a High-level Synthesis (HLS) tool, but that, by itself, cannot lead to an efficient accelerator. This is due to C/C++ code being intrinsically sequential in terms of its computations, which is counter to the high degree of parallelism we can achieve with an efficient hardware implementation.

Both partitioning and optimization have been studied in isolation, but we note a lack of research into the interplay between both processes: one contribution [ZPA⁺], for instance, perform partitioning not by looking at a task as-is, but rather to multiple versions of the same task, each of which using a different set of code optimization recipes. It is a step towards a combined approach, but it still only ultimately looks at each task in isolation, ignoring any optimizations we may perform to a set of tasks, or even how we may optimize each task based on the data it communicates. Other authors [FHP⁺] overcome this by applying optimizations to clusters of tasks, but restrict themselves to basic unrolling and pipelining of loops, and validate it using simple kernels rather than applications with multiple functions.

We propose, then, a holistic approach to partitioning and optimizing a C/C++ application: our process does not consider individual tasks as the target of optimization, but rather clusters of tasks. By having a global view of the application, we can create the clusters along the lines of the data shared between tasks, which helps to minimize the communication cost between the CPU and the FPGA. These clusters then open the door for more and better code optimizations to be applied, as we may apply code transformations between tasks inside the same cluster, or even by implementing efficient communication channels between clusters in different devices. A greedy algorithm then iteratively refines the clusters with the ultimate objective of reducing the overall execution time of the heterogeneous application.

Ultimately, our holistic approach poses whether a single combined approach can achieve better results than those achieved by the sum of the two processes it supplants, as we are now making decisions using data that is not available to either of those processes when applied in isolation. In this poster, we present our work in progress for this contribution, which is still at an early stage: more specifically, we describe the methodology in more detail, and provide some results for a motivational example that help show the potential impact of our approach.

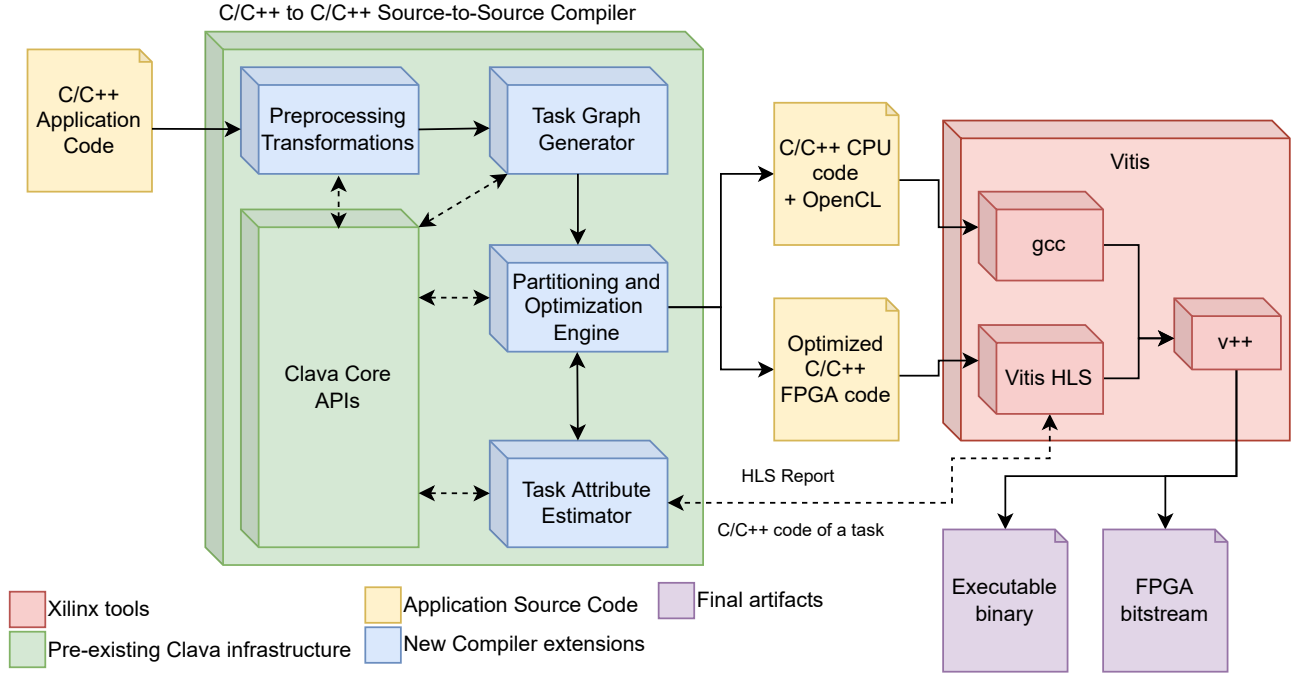


Figure 1: Toolchain used for our holistic partitioning and optimization process

2 Methodology

In order to realize our holistic process, we make use, primarily, of the Clava source-to-source C/C++ to C/C++ compiler [BC]. This compiler allows us to perform code analysis and transformations by using the AST as an intermediate representation, and forms the basis of the toolchain depicted in Figure 1. We extend Clava with a series of steps: firstly, it preprocesses the code to reduce the entropy in its analysis. Then, it builds a hierarchical task graph based on the functions in the source code, so that each task is one uninterrupted sequence of instructions (functions that call other functions are split into multiple tasks at the place of the function calls).

The compiler annotates each task with its FPGA resource usage and estimated latency, which is obtained automatically through the synthesis reports of Vitis HLS, and also a value for the CPU latency, obtained through profiling. The communication cost for each edge is estimated using a linear model that scales with the size of the communicated data.

After we have this task graph, we can begin with the holistic algorithm proper. This algorithm first performs a baseline partitioning by assigning hotspots to the FPGA, and the other regions to the CPU. This provides a set of clusters of tasks, where all tasks inside a cluster are assigned to the same device. Then, it iteratively refines these clusters, moving tasks in and out of them as necessary. As new tasks enter the cluster, it opens the possibility for applying more code optimizations.

These optimizations may be performed at several levels: firstly, there is the communication between clusters, where we can try to create opportunities for FIFOs to be used, as opposed to copying all data at once; secondly, we have optimizations surrounding the communication of tasks inside a cluster: we can, for instance, select tasks for the cluster with a producer-consumer relationship, so that we may enable the creation of dataflow regions; thirdly, we may enable optimizations surrounding the tasks themselves, as we may merge

tasks together into larger tasks. This is useful when we have tasks with similar implementations, and that differ only in the input; and finally we may enable better optimizations inside each individual task, as we may be able to have more efficient memory regions (e.g., by introducing local buffers, and avoiding having to write to external memory).

All of these possible optimizations depend not only on which tasks are on the cluster, but also on how the data communicated between tasks can be rearranged. This requires having a global view of the program, and is something exposed in our task graph formulation. The governing heuristic of our holistic process is, then, to refine the clusters by giving priority to tasks that share high volumes of data, even if they may, at first, seem like a suboptimal decision for lowering the overall execution time. The newly created optimization opportunities can then be applied to improve this initially suboptimal solution.

3 Early Results and Conclusion

In this extended abstract, we presented our proposal for a holistic process to partitioning and optimizing applications in CPU-FPGA heterogeneous systems. By simulating this process using an edge detection application, we can achieve a latency speedup of over $28.7\times$ that of a naïve partitioning that offloads the entire application to an FPGA, with basic loop unrolling and pipelining optimizations. We present this motivating example in our full poster, and portray the different kinds of holistic partitioning and optimization decisions we can apply to it.

As this contribution is a work-in-progress at an early stage, we have, at the time of writing, begun consolidating the infrastructure. More specifically, we have implemented the seamless and automatic integration of the Clava compiler with the Xilinx tools, as well as the preprocessing transformations, and are currently on the process of implementing a robust task graph generator. The next immediate steps are to statistically study how communication of data between tasks evolves throughout each program's execution, by using several large applications. This information can then be used to jumpstart the design of our holistic partitioning and optimization process.

References

- [BC] João Bispo and João M.P. Cardoso. Clava: C/C++ source-to-source compilation using LARA. 12:100565.
- [FHP⁺] Gereon Fuhr, Seyit Halil Hamurcu, Diego Pala, Thomas Grass, Rainer Leupers, Gerd Ascheid, and Juan Fernando Eusse. Automatic Energy-Minimized HW/SW Partitioning for FPGA-Accelerated MPSoCs. 11(3):93–96.
- [HYH] Neng Hou, Xiaohu Yan, and Fazhi He. A survey on partitioning models, solution algorithms and algorithm parallelization for hardware/software co-design. 23(1-2):57–77.
- [ZPA⁺] Wei Zuo, Louis-Noel Pouchet, Andrey Ayupov, Taemin Kim, Chung-Wei Lin, Shinichi Shiraishi, and Deming Chen. Accurate High-level Modeling and Automated Hardware/Software Co-design for Effective SoC Design Space Exploration. In *Proceedings of the 54th Annual Design Automation Conference 2017*, pages 1–6. ACM.